

ENCONTRO 3: Busca Linear e Atualização de Dados

Objetivo: Criar o motor de busca sequencial, utilizar o acesso direto aos dados para modificar registros existentes.

Instrução: O Scrum Master tem 10 minutos para focar no algoritmo de varredura e no debate teórico. O Arquiteto documenta os fluxos e os principais tópicos da discussão.

Fatos (Vamos definir os consensos):

O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT e o Arquiteto documenta.

1. **Parâmetros e Retorno da Busca:** O que a OAT define exatamente que o método estático de busca linear deve receber como entrada e o que ele deve retornar após varrer a lista?
 - **Fato Extraído:** O método estático de busca linear recebe como entrada o código numérico da peça que deseja encontrar. Ele percorre o array de peças de modo sequencial e retorna um int, indicando a posição em que a peça foi encontrada.
2. **A Operação de Atualização:** Qual é a definição exata de Update (Atualização) cobrada pela OAT para os dados existentes?
 - **Fato Extraído:** Consiste em localizar uma peça existente pelo seu código e modificar seus dados, como nome, fabricante, quantidade, preço de venda e preço de custo.
3. **Análise de Limitações:** Um dos objetivos da Sprint é que a equipe saiba analisar as características e limitações práticas da estrutura. Baseado no Fato do Encontro 1, qual é o limite físico fixado para o array de Peças?
 - **Fato Extraído:** A limitação é que o tamanho deve ser previamente definido. O array de peças possui capacidade máxima de 100 elementos, com índices de 0 a 99.



Brainstorm Investigativo (Pesquisem e debatam!)

1. **O Motor de Busca:** A OAT exige que o método de busca retorne um "indicativo de erro" caso a peça não exista. Sabendo que índices de arrays em Java vão de 0 em diante, qual número inteiro seguro podemos retornar para representar que o item não foi encontrado?

Decisão da Equipe

O número inteiro seguro utilizado será -1

Como as posições válidas de um array começam sempre em 0 e vão até tamanho - 1. O uso do número -1 é a forma padrão e segura.

2. Integração e Reuso (DRY): Precisamos encontrar a peça antes de atualizá-la. Como podemos projetar o método de Atualização (Update) para que ele chame o método de Busca Linear lá dentro, em vez de reescrevemos um novo laço de repetição (for) só para a atualização?

- **Decisão da Equipe:** Foi decidido que o método de atualização deve chamar o método de busca linear, passando o código da peça que será atualizada. A busca retorna a posição da peça. Se o retorno for diferente de -1, o método utiliza essa posição para acessar diretamente o objeto no array e alterar seus atributos através dos setters. Caso retorne -1, a peça não existe e a atualização não é realizada.

3. A Defesa do Limite (Preparação para Apresentação): Se o limite do array é 100, o que acontece com a memória, com o programa Java e com o negócio da MecâniQA se o usuário tentar cadastrar a 101ª Peça? Como explicaremos essa desvantagem arquitetural para o professor na avaliação oral?

- **Decisão da Equipe:** Como o array possui capacidade fixa de 100 peças, quando tentar cadastrar a 101ª peça, não existe mais posição disponível no array. O programa não aumenta de forma automática a sua capacidade, logo o novo cadastro deve ser recusado por atingir o limite. A limitação é principalmente de capacidade do sistema, não significa que o Java ficará sem memória simplesmente por tentar cadastrar a 101ª peça.

Entrega do Encontro 3n

- **Código:** O Piloto deve implementar o método estático de Busca Linear e o método de Atualização na classe `Gerenciador.java`.

```
public static void updateServico(int codigoAlvo, String novaDescricao, int novoTempoEstimado, double novoValorMaoObra) {
    for (int i = 0; i < servicos.length; i++) {
        if (servicos[i] != null && servicos[i].getCodNumerico() == codigoAlvo) {
            servicos[i].setDescricaoServico(novaDescricao);
            servicos[i].setTempoEstimado(novoTempoEstimado);
            servicos[i].setValorMaoObra(novoValorMaoObra);
            System.out.println("Serviço atualizado com sucesso!");
            break;
        }
    }
}

public static void buscaLinearServico(int codigoBuscado) {
    for (int i = 0; i < servicos.length; i++) {
        if (servicos[i] != null && servicos[i].getCodNumerico() == codigoBuscado) {
            System.out.println("ServiçoIndex = " + i + "\n " + "Codigo do serviço: " + servicos[i].getCodNumerico());
            if (codigoBuscado < 0 || codigoBuscado > servicos.length) {
                System.out.println("Erro ao buscar serviço");
                return;
            }
        }
    }
}
}
```

```
public static void updatePeca(int codigoAlvo, String novoNomePeca, String novoNomeFabricante, int novaQuantidadePeca,
    double novoPrecoVenda, double novoPrecoCusto) {
    for (int i = 0; i < pecas.length; i++) {
        if (pecas[i] != null && pecas[i].getCodNumerico() == codigoAlvo) {
            pecas[i].setNomePeca(novoNomePeca);
            pecas[i].setNomeFabricante(novoNomeFabricante);
            pecas[i].setQuantidadePeca(novaQuantidadePeca);
            pecas[i].setPrecoVenda(novoPrecoVenda);
            pecas[i].setPrecoCusto(novoPrecoCusto);
            System.out.println("Peça atualizada com sucesso!");
            break;
        }
    }
}

public static void buscaLinearPeca(int codigoBuscado) {
    for (int i = 0; i < pecas.length; i++) {
        if (pecas[i] != null && pecas[i].getCodNumerico() == codigoBuscado) {
            System.out.println("PecaIndex = " + i + "\n " + "Codigo da peça: " + pecas[i].getCodNumerico());
        }
    }
}
}
```

- **Apresentação Final:** Estruturação dos slides ou do discurso de defesa respondendo aos questionamentos técnicos do Diretor.



TC-09 - Busca Linear de Peca existente

ID: TC-09

Responsavel (QA): Victor Paulo

Data/Hora: 11/08/2026 - 22:30

O que está testando: localização de uma peça pelo código numérico.

Cenario Dado: uma peça com codNumerico = 1 cadastrada, quando buscaLinearPeca(1) e chamado,

Então o índice e código da peça devem ser exibidos

STATUS: [PASSOU]

Observação: o método localiza e exibe o resultado corretamente. Porém, continua varrendo o array

Após encontrar o item (sem return), e não avisa o usuário quando o item não existe.

TC-10 - Busca linear de peça inexistente (feedback de erro)

ID: TC-10

Responsavel (QA): Victor Paulo

Data/Hora: 11/08/2026 - 22:30

O que está testando: Comportamento da busca quando a peça não existe no array

Cenário: Dado que nenhuma peça com codNumerico = 99 foi cadastrada, quando buscaLinearPeca(99)

É chamado, então o sistema deve informar que o item não foi encontrado

STATUS: [NEGADO]

Causa do erro: o método retorna void e falha silenciosamente. Quando o código não é encontrado,

O for encerra e nada é impresso na tela.

Alteração sugerida:

```
public static int buscaLinearPeca(int codigoBuscado) {  
    for (int i = 0; i < pecas.length; i++) {  
        if (pecas[i] != null && pecas[i].getCodNumerico() == codigoBuscado) {  
            return i;  
        }  
    }  
    System.out.println("Erro: peça não encontrada.");  
    return -1;  
}
```

TC-11 - Busca Linear de Serviço existente

ID: TC-11

Responsável (QA): Victor Paulo

Data/Hora: 11/08/2026 - 22:30

O que está testando: localização de um serviço pelo codigo numerico

Cenario: Dado um serviço com codNumerico = 101 cadastrado, quando buscaLinearServico(101)

É chamado, então o sistema deve localizar e exibir o serviço corretamente

STATUS: [NEGADO] - BUG CRITICO

Causa do erro: o método possui uma validação logicamente impossível inserida dentro do bloco de sucesso:

```
if (codigoBuscado < 0 || codigoBuscado > servicos.length) {  
    System.out.println("Erro ao buscar servico");  
    return;  
}
```

Esta linha compara o ID do serviço (ex.: 101) com o tamanho do array (50).

Como $101 > 50$ é verdadeiro, o sistema encontra o serviço e, na mesma linha, dispara um erro,

Abortando com resultado contradatorio.

Alteração sugerida:

```
public static int buscaLinearServico(int codigoBuscado) {  
    for (int i = 0; i < servicos.length; i++) {  
        if (servicos[i] != null && servicos[i].getCodNumerico() == codigoBuscado) {  
            return i;  
        }  
    }  
    System.out.println("Erro: Serviço não encontrado.");  
    return -1;  
}
```



TC-12 - Retorno da Busca Linear (tipo int com indicativo de erro)

ID: TC-12

Responsável (QA): Victor Paulo

Data/Hora: 11/08/2026 - 22:30

O que está testando: a busca deve retornar um int para ser reutilizada por outros métodos

Cenário: o método buscaLinear deve retornar o índice do item se encontrado,
ou -1, caso contrario

STATUS: [NEGADO]

Causa do erro: ambos os métodos de busca retornam void. Isso impede que sejam reutilizados

pelos métodos de atualização (update), forçando duplicação de código.

Alteração sugerida: alterar a assinatura para public static int buscaLinearPeca(...)

Retornando i quando encontrar e -1 quando não encontrar.

TC-13 - Atualização de peça reutiliza a busca linear (DRY)

ID: TC-13

Responsável (QA): Victor Paulo

Data/Hora: 11/08/2026 - 22:30

O que está testando: o método updatePeca deve usar buscaLinearPeca internamente, sem repetir o laço for

Cenário: Dada uma peça cadastrada, quando updatePeca é chamado,

Ele deve delegar a localização à buscaLinearPeca antes de atualizar

STATUS: [NEGADO]

Causa do erro: o método updatePeca repete um laço for idêntico ao da busca, violando o princípio DRY (Don't Repeat Yourself).

Alteração sugerida:

```
public static void updatePeca(int codigoAlvo, ...) {  
    int index = buscaLinearPeca(codigoAlvo);  
    if (index != -1) {  
        pecas[index].setNomePeca(novoNomePeca);  
        // ...restante dos setters  
    }  
}
```

TC-14 - Atualização de Serviço reutiliza a Busca Linear (DRY)

ID: TC-14

Responsável (QA): Victor Paulo



Data/Hora: 11/08/2026 - 22:30

O que está testando: o método `updateServico` deve usar `buscaLinearServico` internamente, sem repetir o laço `for`

Cenário: Dado um serviço cadastrado, quando `updateServico` é chamado, ele deve delegar a localização a `buscaLinearServico` antes de atualizar

STATUS: [NEGADO]

Causa do erro: mesma violação do princípio DRY do TC-13.

O `updateServico` duplica o laço de busca desnecessariamente.

Alteração sugerida: mesma abordagem do TC-13, delegando a localização para `buscaLinearServico(codigoAlvo)`.

Componentes:

Victor Paulo Teixeira da Silva e Silva

Luis Filipe Silva Lima

Guilherme Pereira Barreto

Brendha Maria Costa Seixas

Matheus Santos Costa

Agson Asafe

Começamos com a leitura do material, o alinhamento das responsabilidades e a interpretação dos requisitos da OAT. A dinâmica foi estruturada a partir da distribuição dos papéis: Maria atuou como Scrum Master conduzindo a leitura das perguntas,

Matheus assumiu como piloto na parte prática e, como Agson estava sem computador no momento, contamos com o apoio de copilotos. Nessa dinâmica de desenvolvimento, Luis seguiu orientando Matheus diretamente na confecção do material, enquanto Victor Paulo, Maria e Guilherme ficaram responsáveis pela organização e estruturação de todo o material da OAT, garantindo a documentação dos consensos e a qualidade das entregas.

Com a equipe alinhada, partimos para a resolução e definimos os pontos principais da sprint de forma clara. O método de busca linear recebe o código da peça, percorre o array de forma sequencial e retorna o índice onde o item foi encontrado em formato de número inteiro, ou menos um caso não exista. Para a operação de atualização aplicando o princípio DRY, definimos que a função localiza a peça para alterar dados como nome, fabricante, quantidade e preços chamando diretamente a busca linear já existente, evitando a criação de um novo laço de repetição; assim, se a busca retornar um índice válido, os atributos são modificados via setters, e se retornar menos um, a operação é interrompida. Por fim, na defesa do limite do array, pontuamos que a estrutura possui capacidade fixa de cem elementos, cobrindo os índices de zero a noventa e nove. Dessa forma, caso ocorra a tentativa de cadastrar a centésima primeira peça, a ação é recusada por atingir o teto de armazenamento, evidenciando uma limitação arquitetural e de regra de negócio da estrutura de dados estática, e não uma falta de memória física do Java.

Na etapa de validação, Victor Paulo elaborou e executou os casos de teste como QA. No geral, os testes apontaram reprovações nos métodos originais devido a problemas como retornos em `void` que impediam o reuso de código, ausência de mensagens de erro para itens não encontrados, validações lógicas incorretas e duplicação de laços `for` nas operações de atualização, violando o princípio DRY. A partir desses diagnósticos levantados nos testes, a equipe realizou todas as correções necessárias no código, ajustando as assinaturas dos métodos para retornarem `int` com indicativo de erro (`-1`) e integrando a busca diretamente aos métodos de atualização.